

KoiCloud — Propuesta de Proyecto

Proyecto Final — Ingeniería de Software I (2026), Universidad Rafael Landívar

Categoría: Base de Datos como Servicio (DBaaS)

Versión: 4 — 12 de septiembre de 2026

Equipo KoiCloud: Hugo Escobar, Jason Gutiérrez, Jousé Menendez, Diego Joachin

Motivo de la revisión: alcance sellado incluye Web + CLI + MCP como adaptadores delgados; seguridad de agentes y auto-curación siguen fuera / V2

1. Resumen ejecutivo

KoiCloud es una plataforma DBaaS: un desarrollador se registra, contrata un plan con pago simulado y obtiene una instancia PostgreSQL gestionada (*pond*) sin operar infraestructura.

Compromiso del semestre: interfaz **web**, **CLI** y **servidor MCP** como fachadas delgadas sobre un control plane FastAPI y un node-agent que aprovisiona contenedores Docker reales en un VPS. Incluye consola SQL, respaldos básicos, panel de administración, **doble confirmación** en mutaciones CLI/MCP, gate mínima de acceso para el endpoint de agente (demo de aula) y un **guion de demo** con fallback si el LLM falla.

No forma parte del compromiso: auto-curación como producto, validación de hipótesis comerciales con usuarios reales, IaC declarativa, ni seguridad completa de agentes (OAuth, scopes, auditoría) — eso es **V2**.

Tono: showcase de capacidad técnica (misma API, tres clientes). No se afirma ser una “plataforma agéntica”.

2. Problemas que sí resolvemos

#	Problema (operación, verificable en el semestre)	Evidencia
1	Crear Postgres exige conocimiento de ops	URI utilizable tras “crear pond”
2	El estado del servicio es opaco	Panel / CLI / MCP con estado del contenedor
3	Suscripción y pago académicos	Flujo simulado + historial + factura
4	No hay cliente SQL a mano	Consola web / tool MCP con resultados
5	Operar desde chat/CLI sin duplicar la API	Thin adapters + confirmación en dos pasos

Hipótesis de negocio (fuera de lo demostrable)

El hueco de precio frente a ~USD 25/mes y la factura con NIT orientan el catálogo, pero **requieren usuarios reales**. Quedan como hipótesis, no como objetivos del proyecto.

3. Catálogo de servicios

Plan	Descripción	Precio	Vigencia
Sandbox	Uso interno / pruebas del producto	USD 0	10 minutos
Micro	1 pond, 1 GB, backups 7 días	USD 5.00 / mes (IVA incluido)	Mensual
Pro	Hasta 10 ponds, post-pago	USD 0.02 / h + almacenamiento	Por uso

Pago: **simulado** (enunciado). Selección de plan, factura con desglose de IVA, historial, renovación y cancelación: implementados.

4. Alcance comprometido (sin niveles)

No hay núcleo / deseable / ambicioso. Esta es la lista completa:

1. Auth: registro, verificación de correo, recuperación, JWT+refresh; roles Cliente y Administrador.
2. Suscripciones: catálogo, contratación simulada, historial, renovación, cancelación.
3. DBaaS: crear / estado / conexión / eliminar PostgreSQL 16 en Docker.
4. Consola SQL en el navegador (lectura por defecto).

5. Respaldos diarios y restauración bajo demanda.
6. Vista de uso (horas y almacenamiento).
7. Panel administrador.
8. Arquitectura control plane + node-agent + cola en PostgreSQL; un nodo.
9. Web + CLI + MCP thin clients sobre la misma API.
10. Doble confirmación en mutaciones CLI/MCP.
11. Auth mínima de agente (URL + password) para demo.
12. Guion de demo MCP + fallback.

Fuera de alcance

- IaC declarativa (`ccloud.yaml` / `apply`).
- Seguridad completa de agentes = **V2** (OAuth 2.1, scopes, spend caps, auditoría, 2FA).
- Auto-curación como garantía o demo de caos.
- Validación empírica de precio/NIT.
- Otros motores, branching, réplicas, pooler, extensiones.
- Organizaciones/equipos.
- Cobro real, pasarelas, SAT, SLA.
- HA, multi-nodo.
- Roles Soporte / Operador; status page pública; sandbox público sin registro.

5. Arquitectura (defendible con el conocimiento del equipo)

Monolito modular FastAPI que escribe estado deseado; node-agent ejecuta Docker y reporta estado observado; cola de jobs en PostgreSQL (SKIP LOCKED). Un VPS. Web SPA, CLI Typer y FastMCP montado en la API invocan la misma capa de comandos.

Descartado: microservicios; Kubernetes; OAuth de agentes en v1; tratar MCP como producto de seguridad.

Detalle de diagramas: paquete docs/entrega-2/.

6. Stack

React + Vite + TypeScript · CLI Typer · FastMCP · Python 3.12 + FastAPI · PostgreSQL 16 · Docker · Caddy · JWT/argon2 · gate password para agente · Resend o console para correo.

7. Metodología y cronograma

Scrum académico en sprints alineados a los hitos del curso. El Equipo KoiCloud prioriza el camino crítico (auth → ponds reales → billing → consola → CLI/MCP thin → docs/demo).

Hito	Fecha	Objetivo
Propuesta	28 ago	v1
Requisitos y diseño	11 sep	Entrega 2 (este alcance)
Avance 30 %	25 sep	Auth + planes + 1 pond real
Avance 50 %	9 oct	Billing + backups + uso
Avance 80 %	23 oct	Consola SQL + admin + CLI/MCP thin + confirm
Final	1ª sem nov	Docs + demo guionada + exposición

8. Riesgos

Riesgo	Mitigación
Coordinación del equipo	Alcance explícito; mock del agente; thin adapters; contratos de API tempranos
VPS cae el día de la demo	Video de respaldo + modo mock
LLM falla en demo MCP	Guion happy-path + replay/fallback sin modelo
Sobreingeniería residual del harness	Entrega 2 y esta propuesta mandan
Exponer Postgres a Internet	Contraseñas fuertes, puertos altos, firewall

9. Por qué este alcance es honesto

Cumple el enunciado (usuarios, pagos simulados, catálogo DBaaS, web, arquitectura defendida) y añade CLI/MCP como **demonstración de la misma API**, con confirmación en dos pasos y gate mínima — sin fingir un producto de seguridad ni auto-curación. Lo que impresiona en la demo —contenedor real, consola SQL y chat → pond— no depende de OAuth ni de caos engineering.